NYC Trip Duration Prediction and MLOps Demonstration

This notebook demonstrates a complete machine learning workflow for predicting NYC trip durations, focusing on key MLOps practices such as data preprocessing, model training with MLflow, data drift simulation, model monitoring with retraining triggers, and model serving using Docker.

Table of Contents

1. Project Objective

The primary goal of this project is to build and evaluate machine learning models to predict trip durations for New York City taxi rides. Beyond just prediction, this notebook emphasizes demonstrating core MLOps concepts:

- **Schema Validation and Data Cleaning:** Ensuring data quality and consistency.
- **Feature Engineering:** Creating relevant features for better model performance.
- **MLflow Integration:** Tracking experiments, logging models, parameters, and metrics.
- **Data Drift Simulation:** Understanding how changes in input data can impact model performance.
- **Model Monitoring:** Detecting performance degradation and setting up automated retraining triggers.
- **Model Packaging and Serving:** Creating a deployable containerized model for production use.

2. Data Loading and Initial Exploration

The dataset, `NY_Trip_Raw_Data.csv`, contains information about NYC taxi trips, including pickup/dropoff times and locations, passenger count, and weather conditions. The initial steps involve loading this data into a Pandas DataFrame and performing basic checks:

- **`df.info()`**: Provides a summary of the DataFrame, including data types and non-null counts.
- **`df.isnull().sum()`**: Checks for missing values in each column.
- **`df.describe()`**: Generates descriptive statistics for numerical columns.

Key Observations:

- The dataset contains 1,048,575 entries.

- No missing values were found in the raw data.
- Datetime columns (`pickup_datetime`, `dropoff_datetime`) are initially loaded as `object` (string) type, requiring conversion.

3. Data Preprocessing and Feature Engineering

This section prepares the raw data for modeling by cleaning it and creating new, informative features.

- **Schema Validation:**
  - `pickup_datetime` and `dropoff_datetime` columns are converted to datetime objects using `pd.to_datetime`.
  - Checks are performed for unparseable dates and illogical timestamps (dropoff before pickup).
  - Rows with invalid timestamps are removed.
- **Feature Engineering:**
  - **`pickup_hour`**: Extracted from `pickup_datetime` to capture time-of-day patterns.
  - **`pickup_dayofweek`**: Extracted to identify day-of-week patterns.
  - **`is_weekend`**: A binary feature indicating if the trip occurred on a weekend (Saturday or Sunday).
  - **`distance_km`**: Calculated using the Haversine formula based on pickup and dropoff coordinates, representing the great-circle distance between two points on a sphere.
  - **`weather`**: Converted to a categorical type for efficiency.

4. Model Training and Comparison

Two regression models are trained and evaluated to predict `trip_duration`:

- **Linear Regression:** A simple, interpretable model.
- **Gradient Boosting Regressor:** A more complex ensemble model

known for its high performance.

Both models are tracked using **MLflow** to log parameters, metrics (Mean Squared Error, R-squared), and the trained models themselves. This allows for easy comparison and reproducibility of experiments.

Data Preparation for Modeling:

- Features (`X`) include engineered features and one-hot encoded categorical variables (`weather`, `store_and_fwd_flag`, `vendor_id`).
- The target variable (`y`) is `trip_duration`.
- The data is split into 80% training and 20% testing sets.
- NaN values in training and testing sets are handled before model fitting and prediction.

Linear Regression

- **Model:** `sklearn.linear_model.LinearRegression`.
- **Training:** Fitted on `X_train_cleaned`, `y_train_cleaned`.
- **Evaluation:** `mean_squared_error` and `r2_score` are calculated on `X_test_cleaned`, `y_test_cleaned`.
- **MLflow Logging:** Model parameters (`model_type`), metrics (`mse`, `r2`), and the model artifact (`linear_regression_model`) are logged.

Performance (Example):

- MSE: ~9,557,237.30
- R-squared: ~0.03

Gradient Boosting Regressor

- **Model:** `sklearn.ensemble.GradientBoostingRegressor`.
- **Hyperparameters:** `n_estimators=100`, `learning_rate=0.1` (example values).
- **Training:** Fitted on `X_train_cleaned`, `y_train_cleaned`.
- **Evaluation:** `mean_squared_error` and `r2_score` are calculated on `X_test`, `y_test`.

- **MLflow Logging:** Model parameters (`model_type`, `n_estimators`, `learning_rate`), metrics (`mse`, `r2`), and the model artifact (`gradient_boosting_model`) are logged.

Performance (Example):

- MSE: ~11,025,336.11
- R-squared: ~-0.12

Conclusion: Based on the R-squared values, the Linear Regression model appears to perform slightly better than the Gradient Boosting Regressor in this initial comparison.

5. Data Drift Simulation and Monitoring

This section simulates a common real-world scenario: **data drift**, where the characteristics of the input data change over time, potentially degrading model performance. It then sets up a monitoring mechanism to detect such degradation and trigger a retraining process.

Simulating a 'Rush Hour Surge' Data Drift

- A copy of the original DataFrame (`df_drift`) is created.
- A

rush hour surge

is simulated by: * Increasing `passenger_count` for a percentage of trips during defined rush hours. * Slightly increasing `distance_km` and `trip_duration` for rush hour trips to mimic congestion.

- The drifted data (`X_drift`, `y_drift`) is then prepared with the same feature engineering steps as the original data, ensuring column consistency for model prediction.

Evaluating Performance on Drifted Data

The previously trained Linear Regression model is used to make predictions on the `X_drift` dataset. The model's performance (MSE, R-squared) on this drifted data is then compared against its original performance.

Observation: A significant decrease in R-squared is observed on the drifted data, indicating performance degradation.

Setting Up Monitoring and Retraining Trigger

- A performance degradation threshold is defined (e.g., a 10% drop in R-squared from the original baseline).
- If the model's performance on the drifted data falls below this threshold, a retraining process is triggered.
- **Simulated Retraining:** The Linear Regression model is re-trained using the original clean training data (in a real-world scenario, this would involve new, more representative data).
- The retrained model's performance is re-evaluated, showing a recovery in R-squared.
- The retrained model is logged to MLflow as a new run (`Linear_Regression_Model_Retrained`).

6. Improving Model Performance: Hyperparameter Tuning

Given the initial low R-squared scores, hyperparameter tuning is introduced as a method to optimize model performance. `GridSearchCV` is used to systematically search for the best combination of hyperparameters for the `GradientBoostingRegressor`.

- **Parameter Grid:** Defines a range of values for `n_estimators`, `learning_rate`, and `max_depth`.
- **`GridSearchCV` Setup:** Configured with 3-fold cross-validation (`cv=3`), using all available CPU cores (`n_jobs=-1`), and optimizing for R-squared (`scoring='r2'`).

- **Fitting:** `GridSearchCV` is fitted to the cleaned training data (`X_train_cleaned`, `y_train_cleaned`).
- **Results:** The best parameters and the best R-squared score from cross-validation are identified.
- **MLflow Logging:** The tuned model's parameters and performance metrics are logged to MLflow as `Gradient_Boosting_Tuned_Model`.

7. Model Packaging and Serving

This section demonstrates how to prepare the best-performing model (Linear Regression in this case) for deployment by packaging and serving it as a REST API.

Demonstrate Prediction with Loaded Model

- The Linear Regression model is loaded from MLflow using its run ID.
- A sample of `X_test` data is used to demonstrate how to make predictions with the loaded model.
- Predictions are printed alongside actual values for comparison.

Serve Model via REST API using MLflow (Manual)

Instructions are provided on how to serve the logged MLflow model as a local REST API using the `mlflow models serve` command from the terminal.

- **Command Structure:** `mlflow models serve -m runs:/<your_linear_regression_run_id>/linear_regression_model -p 5000`.
- **JSON Payload Example:** A sample JSON payload is provided for sending prediction requests to the API endpoint (`http://127.0.0.1:5000/invocations`).

Generate Dockerfile for Self-Contained Serving

To create a truly portable and deployable model, a Dockerfile is generated. This Dockerfile creates a self-contained image that includes the model artifacts and all necessary dependencies, allowing it to be served without relying on an external MLflow tracking server at runtime.

- **Steps:**
    1. The model artifacts for the Linear Regression model are downloaded locally from MLflow.
    2. A `requirements.txt` file is generated, listing all Python dependencies.
    3. A `Dockerfile` is created:
        - Uses a Python base image (`python:3.9-slim-buster`).
        - Copies `requirements.txt` and installs dependencies.
        - Copies the downloaded model artifacts into the image.
        - Exposes port 5000.
        - Defines the `CMD` to serve the model using `mlflow models serve` from the local path within the container.
- **Docker Build/Run Instructions:** Commands for building and running the Docker image locally are provided, along with important notes about `MLFLOW_TRACKING_URI` for remote MLflow servers.

8. MLflow UI Access

The notebook includes a cell that starts the MLflow UI in the background and creates an ngrok tunnel to expose it to a public URL. This allows easy access to the MLflow experiment tracking interface from any web browser to view runs, compare models, and inspect artifacts.

- **Steps:**
    - Installs `pyngrok`.
    - Kills any existing ngrok tunnels.

    - Starts `mlflow ui --port 5000` in the background.
    - Connects ngrok to port 5000 and prints the public URL.
- **Note:** An ngrok authentication token is required.

9. Resources

This section can be used to list various resources related to this project, such as:

- **Data Sources:** Links or descriptions of the datasets used.
- **Code Repositories:** Links to GitHub or other version control systems.
- **Documentation:** Links to external documentation, research papers, or articles.
- **MLflow UI Link:** The ngrok tunnel URL for the MLflow UI (e.g., `NgrokTunnel: "https://<your_ngrok_id>.ngrok-free.app" -> "http://localhost:5000"`).
- **Deployment Information:** Details about where the model is deployed or how to access it